

Disclaimer: Dieses OER-Dokument stammt aus dem Projekt [proTirone](#), das über GitHub [freie Unterrichtsmaterialien](#) samt [Quellcode](#) offeriert. proTirone-Materialien werden unter den Bedingungen der [CC-BY-4.0-Lizenz](#) so angeboten, wie sie sind, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung (§5). Dafür dürfen sie – bei angemessener Namensnennung (§3) – für beliebige (auch kommerzielle) Zwecke verändert und weitergegeben werden (§2).

LFCX: py2fr = Kleiner Python Refresher

1. Ausgangspunkt [→ ZP:Sheet:02]

- Auflösung der LF09/09-Aufgabe: Was tut dieses Programm [→ ZP:Sheet:02]?
 - Netzadresse wird aus IP-Adresse und Netzmaske per bitweiser &-Verknüpfung abgeleitet.
 - Dies verifiziert das Programm anhand eines Sets korrekter Netzspezifikationen.
- Unterrichtserfahrung: Gelegentlich steigen Schülerinnen (Männer sind wie immer mitgemeint) beim Lesen des Programms aus. Auf Nachfrage:
 - Sie wollten das Programmieren doch erst in der Lehre lernen.
 - Hätten aber zu wenig Zeit und Möglichkeiten dazu.
 - Ungerecht: Ein Elektrolehrling müsse doch vor der Lehre auch nicht schon ein ganzes Haus verkabelt haben.
- Abhilfe:
 - Für die, denen (Python-) Programmierung ganz fremd ist, stelle ich `__als Einstieg__` → **cx.py2go** zur Verfügung: (Ziel: lerne nebenbei, 10 min. am Tag).
 - Für die anderen biete ich einen kleinen Python-Refresh-Kurs **cx.py2fr** (Ziel: Das Wesentliche auf einen Schlag)

2. Kleine Fingerübungen:

Vorabinfos:

- Python3-Installation:
 - LNX/Ubuntu: `sudo apt-get install python3`
 - W11:
 - * *Python install manager* aus dem Microsoft Store installieren & ausführen.

* → <https://docs.python.org/dev/using/windows.html>

ÜBUNG LFCX:py2fr:01

- ☐ Bringen Sie ein Python-Hello-World auf Ihrem Rechner zum Laufen. [→ ZP:Sheet:03]

(→ `ex-py2fr-a-hello-world.py`)

ÜBUNG LFCX:py2fr:02

- ☐ Bringen Sie das Programm von [→ ZP:Sheet:02] auf Ihrem Rechner zum Laufen.

(→ `ex-py2fr-b-nw-validation.py`)

3. Verschiedene Tests der Netzwerkzugehörigkeit [→ ZP:Sheet:04]

Hinweis: Es gibt verschiedene Lösungswege. Deshalb: *meine Lösungen* als Varianten

ÜBUNG LFCX:py2fr:03

- ☐ Schreiben Sie mit den Mitteln aus obigem Programm [→ ZP:Sheet:02] eine Funktion / Programm,
 - ☐ die eine IPv4-Adresse samt einer zu ihr passenden Netzmaske und eine beliebige andere IPv4-Adresse nimmt
 - ☐ diese von Kommandozeilenparametern überschreiben lassen kann
 - ☐ und die berechnet, ob beide Adressen in demselben Netz sind

Nutzen Sie bei der Berechnung **Bitweise Operatoren**, nicht fertige Abfragefunktionen aus Bibliotheken wie z.B. `ipaddress`. Sie dürfen aber gern deren andere Funktionen nutzen, um IPv4-Strings in Zahlen (und v.v.) umzuwandeln.

Meine Lösung: [→ ZP:Sheet:05]

Anmerkungen:

- Bibliotheken / Module binde ich eingangs per **import**-Statement ein - i.d.R. per Dateinamen.
- Klassen, Variablen, Funktionen etc. aus den Bibliotheken referenziere ich per dot-Notation (Z8, Z14)

- Das Betriebssystem übergibt jedem Programm Kommandozeilenparameter als ein Vektor von Argumenten `argv[]`.
- An der ersten Stelle (`argv[0]`) wird immer der Programmname übergeben.
- An der zweiten bis n-ten Stelle (`argv[1] ... argv[n]`) befinden sich die einzelnen Kommandozeilenargumente.
- Beispiel:
 - # `python3 myscript.py arg1 arg2 arg3` erhält `[python3, myscript.py, arg1, arg2, arg3]`
 - `python3` macht daraus für `myscript.py`: `[myscript.py, arg1, arg2, arg3]`
- C/C++-Programmierer müssen diese noch von Strings in die intendierten Typen umwandeln. Python-Programmierer kriegen das geschenkt.

Konzeption:

- Eine IPv4-Adresse mit ihrer Netzwerkmaske per binärem ‘&’ verknüpft, ergibt die Netzadresse
- Werden zwei IPv4-Adressen mit derselben Netzwerkmaske per binärem ‘&’ verknüpft und ergeben jeweils dieselbe Netzadresse, sind sie in demselben Netz.

ÜBUNG LFCX:py2rf:04

- ☐ Fügen Sie eine logische Bedingung ein, sodass der Test nur gestartet wird, wenn die beiden IPv4-Adressen nicht gleich sind.

Meine Lösung: [→ ZP:Sheet:06]

Anmerkungen:

- Bibliotheken/Module können auch in einer Zeile importiert werden. (Z2)
- Ähnlich können mehrere Variablen in einer Zeile ‘auf einen Schlag’ initialisiert werden. (Z4)
- Das logische `and` operiert auf Wahrheitswerten (Z16), der Bitweise Operator `&` auf Bits (Z12).
- In Z16 habe ich aus Demo-Gründen eine redundante Bedingung eingefügt. Prinzipiell ist das unschön, schadet aber nicht:
 - *if-Klauseln* werden gemäß der Wahrheitswertetabelle abgearbeitet.

- Eine logische Konjunktion als Ganzes ist wahr, wenn jede Teilaussage wahr ist.
 - Sobald sich eine Teilbedingung als falsch herausgestellt hat, kann die logische Konjunktion als solche nicht mehr wahr werden.
 - Also: Abbruch der Klauselberechnung.
 - Funktionsblöcke werden in Python durch Einrückungen zusammengefasst (keine Klammern). (Z11, Z17, Z20)
 - Funktionsheader werden mit `:` dem Funktionsblock vorangestellt und eine Ebene weniger eingerückt.
-

ÜBUNG LFCX:py2rf:05

- ☐ Überführen Sie die bisherige funktionale Lösung in eine objektorientierte Lösung (*Schreiben Sie eine Klasse mit Python-Konstruktor und getter- und setter-Methoden*)
 - ☐ Fügen Sie eine Methode `isMember` hinzu, die eine fremde IPv4-Adresse nimmt und `true` oder `false` zurückgibt, je nachdem, ob die übergebene IPv4-Adresse zu dem Netz gehört oder nicht
 - ☐ Erlauben Sie im Main-Teil Ihres Programms, dass die zu testende IPv4-Adresse an der Kommandozeile an Ihr Programm übergeben wird
-

Lösung:

1. eine Klasse zum Speichern und Berechnen (Ableiten) der Netzwerkspezifikation: Ggb. IPv4-Adresse und Subnetzmaske. Ber. Netzadresse, Broadcastadresse und Gatewayadresse. [→ ZP:Sheet:07]
2. ein von 1 abgeleitete Klasse zur Ausgabe / Nutzung der Netzwerkdaten [→ ZP:Sheet:08]
3. ein Mainprogramm zur Nutzung der Klassen [→ ZP:Sheet:09]

Anmerkungen zu [→ ZP:Sheet:07]:

- Klassendefinitionen beginnen mit `class` gefolgt von einer Majuskel + Folgezeichen (Z4)
- Der Corpus der Klassendefinition beginnt mit einer Initialisierungsfunktion `__init__` (`self, ...`) (Z5)
 - die Initialisierungsfunktion wird beim Anlegen eines Klassenobjektes (Z18) automatisch aufgerufen.
 - die Parameterliste der Initialisierungsfunktion beginnt mit einer Referenz auf das Objekt (`self`) und beliebig vielen anderen Parametern. (Z5)

- Methoden-/Funktionsparameter können bei der Definition mit einem Defaultwert angelegt werden, der benutzt wird, wenn beim Aufruf der entsprechende Parameter fehlt.
 - Achtung: Beim Anlegen eines Klassenobjektes werden nur die Parameter außer dem *self* übergeben. Das *self*-Objekt wird vom Interpreter automatisch ergänzt.
 - Auch Membervariablen können in einem Rutsch initialisiert werden (Z6).
 - In dieser Lösung: Adressen als Integer gespeichert (Z7 + Z8).
 - Deshalb Netzadresse direkt per Bitoperator & ableitbar (Z9).
 - Broadcastadresse ist höchste Zahl des Adressbereiches binär verodert mit Netzadresse. Höchste Zahl im Adressbereich = binär-inverse Umdrehung der Subnetzmaske (Z10)
 - Zeile 11: das von Python empfohlene Konstrukt zum Umgehen des Überlaufs.
 - Gatewayadresse = Netzadresse + 1 (könnte auch Broadcastadresse - 1 oder sonst was sein).
- Geforderte **getter**-Funktionen für die Membervariablen (Z14, Z15):
 - Python kennt formal keine privaten oder protected Membervariablen oder -methoden (= klasseneigene Funktionen).
 - * In C/C++/Java können private Membervariablen nur durch *getter* oder *setter*-Methoden modifiziert / ausgelesen werden. Der Compiler bzw. Interpreter würde es nicht erlauben, von außen oder aus inneren Membermethoden direkt auf diese Variablen zuzugreifen.
 - * In C/C++/Java könnten protected Membervariablen von außen nur durch *getter* oder *setter*-Methoden modifiziert / ausgelesen werden, von innen (aus Member-Funktionen) aber direkt.
 - Python hat **private Membervariablen/Funktionen nur per Schreibkonvention**: Member, die mit einem Unterstrich beginnen werden vom Programmierer wie echte private Member behandelt, also nicht per direktem Zugriff verändert.
 - **Disclaimer**: Aus Platzgründen habe ich diese Regel hier nicht durchgehalten: Z19 nutzt die getter-Methoden, Z20 greift direkt auf die 'privaten' Membervariablen zu.
 - Die Klasse **ProTiCo_Network_Data** soll bei einer anderen Klassendefinition importiert werden:
 - Um sie für sich testen zu können, wird mit einer if-Klausel ein 'künstlicher' Main-Bereich angesprochen. (Z17)
 - Getestet wird das Anlegen eines Objektes mit den Defaultdaten und deren Abruf.

- Python kennt keine typisierten Variablen. (Die Typisierung ist dynamisch). Warum ist das ungünstig bei Funktionen/Methoden, deren Parameter in der Definition per Default einen Wert zugewiesen bekommen?

Antwort auf diese “Interimsfrage”:

- Python kennt keine typisierten Variablen. (Die Typisierung ist dynamisch)
 - Deshalb kennt Python auf keine Funktionssignaturen (Abfolgen von Parametertypen)
 - Parameter aus den hinteren Reihen werden einfach nach vorne durchgereicht, wenn vordere fehlen.
 - Deshalb liegt die Verantwortung für einen sauberen Aufruf beim Programmierer
-

Anmerkungen zu [→ ZP:Sheet:08]

- Eine Klasse wird per Namen der übergeordneten Klasse in Klammern von ihr abgeleitet (Z6).
- Die übergeordnete Klasse habe ich als Modul importiert.
- Importierte Bibliotheken / Module (mit langen Dateinamen) dürfen per `as` ‘abgekürzt’ werden (Z4).
- Eine abgeleitete Klasse erbt alle Eigenschaften (Membervariablen) und Fähigkeiten (Methoden) der übergeordneten Klassen: Man kann sie nutzen, als seien sie in der abgeleiteten Klasse selbst definiert.
- Die Initialisierung der übergeordneten Klassen geschieht aus der `__init__`-Funktion der abgeleiteten Klassen heraus (Z9).
- Der geforderte `is_member`-Test steht in Z22

Anmerkungen zu [→ ZP:Sheet:09]

- wendet bekannte Techniken an.
- liest zwei IPv4-Strings ein. (Z8-Z12)
- generiert das entsprechende Network-Objekt. (Z14)
- lässt dessen (bei der Anlage berechnete) Beschreibung ausgeben. (Z15)
- liest einen dritten IPv4-String ein. (Z17)
- lässt überprüfen, ob der zu dem definierten Netz gehört. (Z18-Z20)